

Data Mining practice7 버리

Task 2: Book Analysis

```
book_train <- read.csv("C:/Users/silkj/Desktop/한동대학교/5학기/데이터 마이닝 실습/Data-Mining-Practicum/myR/7week practice/dataset for recommendation watcha/book_train.csv",
                      fileEncoding = "CP949", # 또는 "EUC-K
R"
                      stringsAsFactors = FALSE)
book_tast <- read.csv("C:/Users/silkj/Desktop/한동대학교/5학기/데이터 마이닝 실습/Data-Mining-Practicum/myR/7week practice/dataset for recommendation watcha/book_test.csv",
                     fileEncoding = "CP949", # 또는 "EUC-K
R"
                     stringsAsFactors = FALSE)
```

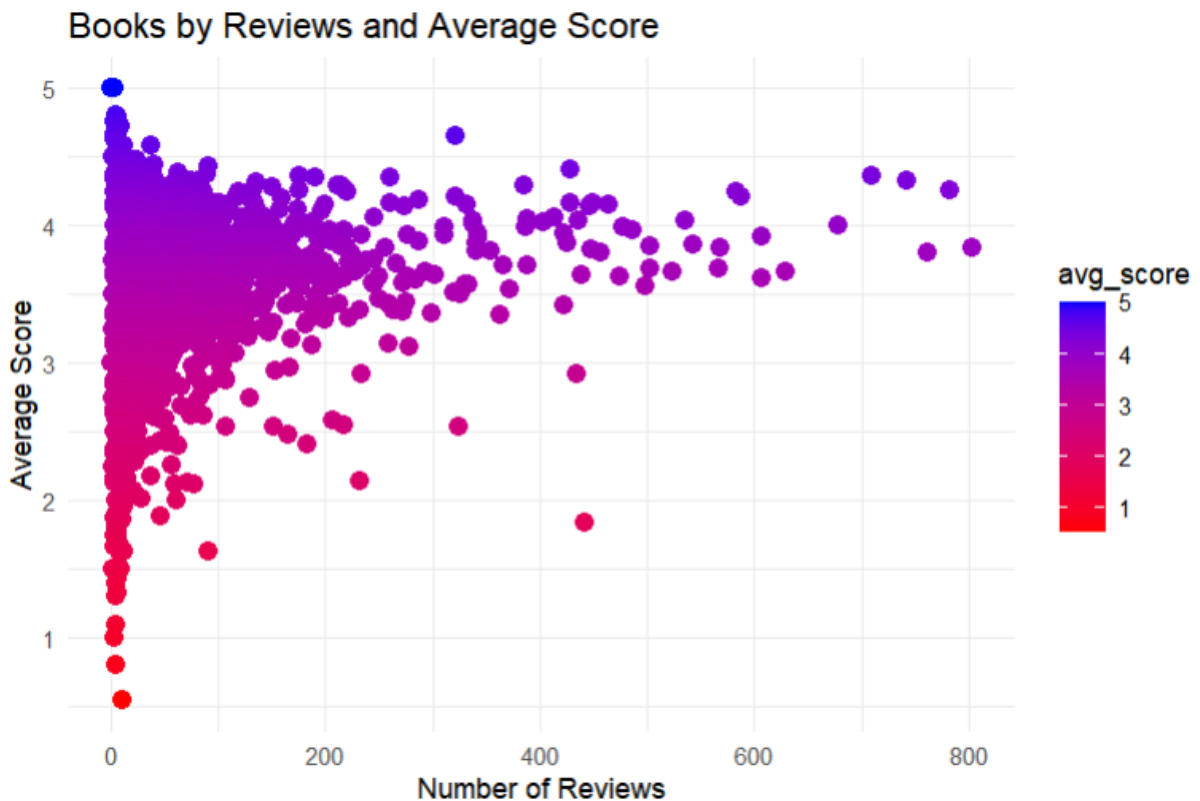
```
book_train <- book_train %>%
  group_by(book_code) %>%
  mutate(reviews = n())

head(book_train)
book_train %>%
  group_by(book_code) %>%
  mutate(avg_score = mean(score)) %>%
  arrange(desc(avg_score))
```

평균 평점과, 각 book별 리뷰의 갯수를 세어 저장함

한번 전체 book에 대해서 평균 점수와 리뷰 수를 시각화해 보았다

```
ggplot(book_train, aes(x = reviews, y = avg_score, label =
book_title)) +
  geom_point(aes(color = avg_score), size = 3) + # 각 책을 점
으로 표시, 평균 평점에 따라 색상 지정
  scale_color_gradient(low = "red", high = "blue") + # 색상
변화 (평균 점수에 따라)
  labs(title = "Books by Reviews and Average Score",
        x = "Number of Reviews",
        y = "Average Score") +
  theme_minimal()
```



상당수의 책들이 리뷰 수가 200개 안쪽에서 많이 이루어지고 있으며, 리뷰 수가 10개가 채 되지 않는 도서가 상당한 것 같다.

평균 평점이 높은 Book 상위 10개

```
book_train %>%
  arrange(desc(avg_score))
  user_code book_code score book_title
reviews avg_score
  <chr>      <chr>      <dbl> <chr>
<int>      <dbl>
1 US000274 BC0011134 5 세월호, 그날의 기록
4 5
2 US000950 BC0018957 5 웅크린 말들
1 5
3 US000183 BC0008295 5 매잡이
5 4.8
4 US000307 BC0012317 5 일방통행로 / 사유이미지
7 4.79
5 US000086 BC0006213 5 A Game of Thrones
4 4.75
6 US001084 BC0023719 5 소설 영웅문 제1부 - 몽고의 별
6 4.75
7 US000159 BC0007898 5 눈물도 빛을 만나면 반짝인다
9 4.72
8 US000130 BC0007430 4 여자는 인질이다
7 4.71
9 US001327 BC0027795 4.5 소설 영웅문 제2부 - 영웅의 별
5 4.7
10 US000291 BC0001481 5 사자왕 형제의 모험
3 4.67
```

review수가 10개가 넘지 못하는 book들이 가장 상단에 위치하여 있다.

이번에는 review의 수를 50개 이상으로 확인해 보고자 한다

```
book_train %>%
  filter(reviews > 50)%>%
  arrange(desc(avg_score))
  user_code book_code score book_title review
s avg_score
  <chr>      <chr>      <dbl> <chr> <int>
```

```

>      <dbl>
1 US000007 BC0001328 3.5 슬램덩크 322
4.66
2 US000014 BC0002526 4 시선으로부터, 91
4.43
3 US000010 BC0001577 5 소년이 온다 429
4.40
4 US000014 BC0002748 5 정확한 사랑의 실험 63
4.39
5 US000018 BC0002244 4 H2 8
9 4.38
6 US000003 BC0000146 5 강철의 연금술사 176
4.36
7 US000003 BC0000359 5 해리 포터와 마법사의 돌 710
4.36
8 US000003 BC0000117 4 여중생A 261
4.35
9 US000008 BC0001044 5 입 속의 검은 잎 191
4.35
10 US000006 BC0001106 4.5 존재의 세 가지 거짓말 82
4.33

```

리뷰 수를 50개 이상으로 하니 다양한 책이 많이 나오는 것 같다. 눈에 띄는 것은 소년이 온다, 해리포터, 슬램덩크 등 많은 사람들이 읽은 인기있는 도서들의 평점이 위로 올라온 것이 보인다.

가장 많은 리뷰가 있는 Book 10개

```

      user_code book_code score book_title      review
s avg_score
      <chr>      <chr>      <dbl> <chr>      <int>
>      <dbl>
1 US000007 BC0000691 4.5 82년생 김지영 803
3.84
2 US000001 BC0000051 4.5 어린 왕자 782
4.25

```

3	US000007	BC0001348	3	나미야 잡화점의 기적	761
3.81					
4	US000003	BC0000351	4	1984	74
2	4.32				
5	US000003	BC0000359	5	해리 포터와 마법사의 돌	710
4.36					
6	US000001	BC0000063	5	데미안	678
4.00					
7	US000003	BC0000314	3.5	채식주의자	630
3.66					
8	US000006	BC0001186	2	노르웨이의 숲	607
3.92					
9	US000003	BC0000346	4	연금술사	607
3.62					
10	US000003	BC0000358	4	해리 포터와 비밀의 방	587
4.21					

이번에는 많은 사람이 읽은 리뷰가 많은 순서대로 정렬하였다. 각각에 대해 평점을 확인해보면 3점대~4점대 사이의 평점을 받고 있음을 알 수 있다.

평점이 높지만 리뷰가 적은 Book 10개

리뷰가 10개 미만인 book에서 평점별로 정렬을 한다

```
top_books <- book_train %>%
  filter(reviews < 10) %>% # 리뷰 수가 적은 책 필터링 (예: 리뷰
가 10 미만인 책)
  arrange(desc(avg_score)) %>% # 평균 평점이 높은 순서로 정렬
  head(10) # 상위 10개 도서 선택
top_books
# Groups:   book_code [10]
  user_code book_code score book_title
reviews avg_score
  <chr>      <chr>      <dbl> <chr>
<int>      <dbl>
  1 US000274 BC0011134 5 세월호, 그날의 기록
4 5
  2 US000950 BC0018957 5 웅크린 말들
```

1	5			
3	US000183	BC0008295	5	매잡이
5	4.8			
4	US000307	BC0012317	5	일방통행로 / 사유이미지
7	4.79			
5	US000086	BC0006213	5	A Game of Thrones
4	4.75			
6	US001084	BC0023719	5	소설 영웅문 제1부 - 몽고의 별
6	4.75			
7	US000159	BC0007898	5	눈물도 빛을 만나면 반짝인다
9	4.72			
8	US000130	BC0007430	4	여자는 인질이다
7	4.71			
9	US001327	BC0027795	4.5	소설 영웅문 제2부 - 영웅의 별
5	4.7			
10	US000291	BC0001481	5	사자왕 형제의 모험
3	4.67			

평점이 낮지만 리뷰가 많은 Book 10개

```
# 평점이 낮고 리뷰가 많은 상위 10개 도서 찾기
bottom_books <- book_train %>%
  filter(avg_score < 3) %>% # 평균 평점이 낮은 도서 필터링 (예:
  # 평점이 3 미만인 책)
  arrange(desc(reviews)) %>% # 리뷰 수가 많은 순서로 정렬
  head(10) # 상위 10개 도서 선택

bottom_books
# Groups:   book_code [10]
  user_code book_code score book_title      rev
  <chr>      <chr>      <dbl> <chr>      <
1 US000001  BC0000013  4   아프니까 청춘이다  44
2          1.84
2 US000003  BC0000190  2.5 마시멜로 이야기  43
```

5	2.92				
3	US000004	BC0000642	2	언어의 온도	3
24	2.54				
4	US000008	BC0000030	0.5	멈추면, 비로소 보이는 것들	234
2.92					
5	US000007	BC0001334	2	시크릿	
232	2.15				
6	US000004	BC0001027	4	하악하악	2
18	2.55				
7	US000004	BC0000886	3.5	오체 불만족	2
07	2.58				
8	US000003	BC0000252	4	지도 밖으로 행군하라	183
2.40					
9	US000003	BC0000240	2	은교	
167	2.96				
10	US000007	BC0001349	0.5	꿈꾸는 다락방	16
5	2.48				

Task5: 도서 추천

UBCF와 IBCF를 통해 train 데이터를 사용해서 test 데이터의 평점을 예측

평점 예측의 MAE(Mean Absolute Error)를 계산하여 보고 얼마나 정확하게 평점을 예측하는지 확인

UBCF

```
#예측하기
# 1. train 데이터와 test 데이터를 결합
combined_data <- rbind(book_train, book_test)
```

```

# 2. test 데이터의 평점을 NA로 설정
combined_data[combined_data$user_code %in% book_test$user_code, "score"] <- NA

# 3. 결합된 데이터를 realRatingMatrix로 변환
combined_matrix <- as(combined_data, "realRatingMatrix")

# 4. UBCF 모델 학습
ubcf_model <- Recommender(combined_matrix, method = "UBCF")

# 5. NA로 설정된 테스트 데이터의 평점을 예측
ubcf_prediction <- predict(ubcf_model, combined_matrix, type = "ratings")

# 6. 예측된 평점을 행렬 형식으로 변환

predicted_ratings <- as(ubcf_prediction, "matrix")

# 7. 테스트 데이터에 해당하는 예측된 평점만 추출
# book_test의 user_code와 book_code에 해당하는 예측 평점만 필터링
test_users <- book_test$user_code
test_books <- book_test$book_code

# 테스트 데이터의 유저와 책 코드에 해당하는 예측 평점을 정확히 매칭하여 추출
predicted_test_ratings <- matrix(NA, nrow = length(test_users), ncol = 1)
for (i in 1:length(test_users)) {
  user <- test_users[i]
  book <- test_books[i]
  predicted_test_ratings[i] <- predicted_ratings[user, book]
}

# 8. 실제 test 데이터에서 평점 추출
# 실제 평점도 test 데이터에 맞게 추출하여 크기 일치
actual_test_ratings <- book_test$score

```



```

# 9. MAE 계산 함수 정의
calculate_mae <- function(predicted, actual) {
  # 예측값(NA)이 아닌 값들만 비교
  valid_idx <- !is.na(predicted) & !is.na(actual)

  # 유효한 비교 값이 없을 경우 NA 반환
  if (sum(valid_idx) == 0) {
    return(NA)
  }

  # 실제 값과 예측된 값의 차이의 절대값을 계산한 후 평균
  mae <- mean(abs(predicted[valid_idx] - actual[valid_idx]))
  return(mae)
}

# 10. 예측된 평점과 실제 평점 간의 MAE 계산 (필터링된 데이터만 처리)
mae <- calculate_mae(predicted_test_ratings, actual_test_ratings)

# 11. 결과 출력
print(paste("MAE:", mae))

```

우선 train데이터와 test 데이터를 병합하였다, 동일한 아이템에 대하여 추천을 진행해야 하는데, train데이터만을 가지고 학습한 후, test데이터를 예측하고자 하면 그 열(아이템)의 수가 일치하지 않기 때문에 제대로 된 추천이 불가능하다.

때문에 train데이터와 test데이터를 병합하며, test데이터의 평점에 대한 값을 NA로 바꾼 후, 모델을 학습시키고. 그렇게 NA값을 예측하게 한다. 이후 실제 test의 예측값과 병합된 train데이터에서의 test부분 예측값을 비교하여 mae값을 계산하는 방법을 사용하였다.

그러나 실제 결과는

```

> print(paste("MAE:", mae))
[1] "MAE: NA"

```

제대로 나오지 않았다. 이 원인을 파악하기 위해

```
> print(sum(is.na(predicted_test_ratings))) # 예측된 평점에서  
NA 개수  
[1] 42454  
> print(sum(is.na(actual_test_ratings))) # 실제 평점에서 NA  
개수  
[1] 0  
> # 예측된 평점과 실제 평점에서 유효한 값 확인  
> print(sum(!is.na(predicted_test_ratings) & !is.na(actual_  
test_ratings))) # 비교 가능한 데이터 수  
[1] 0
```

다음과 같이 NA개수를 세었는데, 병합한 후 매트릭스에서 예측된 평점에서 NA값이 너무 많
아 비교가 불가능하였다.

IBCF

```
# IBCF  
# 1. train 데이터와 test 데이터를 결합  
combined_data <- rbind(book_train, book_test)  
  
# 2. test 데이터의 평점을 NA로 설정  
combined_data[combined_data$user_code %in% book_test$user_c  
ode, "score"] <- NA  
  
# 3. 결합된 데이터를 realRatingMatrix로 변환  
combined_matrix <- as(combined_data, "realRatingMatrix")  
  
# 4. IBCF 모델 학습  
ibcf_model <- Recommender(combined_matrix, method = "IBCF")  
  
# 5. NA로 설정된 테스트 데이터의 평점을 예측  
ibcf_prediction <- predict(ibcf_model, combined_matrix, typ  
e = "ratings")
```

```

# 6. 예측된 평점을 행렬 형식으로 변환
predicted_ratings <- as(ibcf_prediction, "matrix")

# 7. 테스트 데이터에 해당하는 예측된 평점만 추출
test_users <- book_test$user_code
test_books <- book_test$book_code

# 테스트 데이터의 유저와 책 코드에 해당하는 예측 평점을 정확히 매칭하여
# 추출
predicted_test_ratings <- matrix(NA, nrow = length(test_users), ncol = 1)
for (i in 1:length(test_users)) {
  user <- test_users[i]
  book <- test_books[i]
  predicted_test_ratings[i] <- predicted_ratings[user, book]
}

# 8. 실제 test 데이터에서 평점 추출
actual_test_ratings <- book_test$score

# 9. MAE 계산 함수 정의
calculate_mae <- function(predicted, actual) {
  # 결측값(NA)이 아닌 값들만 비교
  valid_idx <- !is.na(predicted) & !is.na(actual)

  # 유효한 비교 값이 없을 경우 NA 반환
  if (sum(valid_idx) == 0) {
    return(NA)
  }

  # 실제 값과 예측된 값의 차이의 절대값을 계산한 후 평균
  mae <- mean(abs(predicted[valid_idx] - actual[valid_idx]))
  return(mae)
}

```

```
# 10. 예측된 평점과 실제 평점 간의 MAE 계산
mae <- calculate_mae(predicted_test_ratings, actual_test_ratings)

# 11. 결과 출력
print(paste("MAE:", mae))
```

동일하게, MAE의 값이 NA로 나왔다.

문제를 하나씩 찾아보자

```
# 1. train 데이터를 realRatingMatrix로 변환
train_matrix <- as(book_train, "realRatingMatrix")

# 2. realRatingMatrix를 일반 행렬로 변환하여 결측값 확인
train_data_matrix <- as(train_matrix, "matrix")

# 3. NA 값의 개수 세기
na_count <- sum(is.na(train_data_matrix))

# 4. NA 개수 출력
print(paste("Number of NA values in train data:", na_count))
```

우선 현재 book_train데이터를 매트릭스로 변환하기만 해도

```
> print(paste("Number of NA values in train data:", na_count))
[1] "Number of NA values in train data: 13677304"
```

무려 천만개 가까이 결측값이 생긴다. 이 상태로는 결코 예측을 할 수가 없다

```
# book_train과 book_test에서 book_code가 동일한 항목을 추출
common_books <- intersect(book_train$book_code, book_test$book_code)
```

```
# 동일한 book_code 개수 세기
common_books_count <- length(common_books)

# 결과 출력
print(paste("Number of common book_code between train and t
est:", common_books_count))
```

book에 대해 train데이터와 test데이터에서 book code가 동일한 항목만을 추출해보니

```
> print(paste("Number of common book_code between train and
test:", common_books_count))
[1] "Number of common book_code between train and test: 533
9"
```

5339개 항목이 나왔다. 물론 이것은 단순히 book code가 하나라도 동일한 경우를 찾은 것이며, train데이터와 test데이터 각각에서 user는 겹치지 않는다.